

# Basic (embeddings as geometry) — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus how a real embeddings pipeline avoids doing unnecessary work). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and the exact pieces

---

### Basic Version

An embedding is just a list of numbers that stands in for a piece of text. Two pieces of text with similar meaning end up with lists of numbers that are "close" to each other.

You need to do three things: get the embeddings for 5 sentences, measure how close every pair of them is, and check that your 2 similar sentences score closest to each other.

Pick your 5 sentences first, on paper, before writing any code: 2 that clearly mean almost the same thing, and 3 that are totally unrelated to those 2 and to each other.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

The exercise has two separate pieces: calling an embeddings API, and computing **cosine similarity** between two vectors.

Cosine similarity measures the angle between two vectors, not their length — this matters because it means a short sentence and a long sentence about the same topic can still score as "close," since only the direction matters, not the size. The formula is:

```
cosine_similarity(a, b) = dot(a, b) / (norm(a) * norm(b))
```

Where `dot(a, b)` is the sum of `a[i] * b[i]` over every position, and `norm(a)` is the length of vector `a` (square root of the sum of its squares).

The exact pieces:

- `client.embeddings.create(model="text-embedding-3-small", input=text)` — the vector is in `response.data[0].embedding`.
- `sum(x * y for x, y in zip(a, b))` for the dot product.
- `math.sqrt(sum(x * x for x in a))` for a vector's norm.
- `itertools.combinations(sentences, 2)` — every unique pair, with no risk of comparing a sentence to itself.

Think about what happens once this stops being a 5-sentence toy script. Calling `client.embeddings.create()` once per sentence, in a loop, is 5 separate network round trips for 5 short sentences — each one carries its own latency, and at real scale (hundreds or thousands of pieces of text) that adds up to a genuinely slow pipeline, one request at a time, for no reason.

The `input` parameter to `embeddings.create()` accepts a *list* of strings, not just one — so all 5 sentences can be embedded in a single API call, and the response comes back as a list of vectors in the same order. This is the same idea as Doc01's "fail at startup, not later," just applied to network calls: do the expensive thing once, in bulk, instead of many times for no benefit.

There's a second thing worth knowing, not just doing: OpenAI's embedding vectors already come back unit-length (`norm(a) == 1.0` for every one of them). That means `cosine_similarity(a, b)` mathematically simplifies to just `dot(a, b)` — the division by `norm(a) * norm(b)` divides by `1.0 * 1.0`, which changes nothing. A real pipeline that calls this same embedding model thousands of times a day skips the two `math.sqrt()` calls entirely and just uses the dot product, once this assumption has actually been checked and confirmed — not assumed blindly, since not every embedding model guarantees unit-length vectors.

The design question underneath both of these: **what's genuinely necessary work, versus work you're doing out of habit?** A general-purpose `cosine_similarity()` that handles any vector, from any model, is the safe default to write first. Once you know exactly which model you're using and that it always returns unit vectors, dropping the redundant normalization is a real, measurable optimization — not premature.

**Difference between Basic and Intermediate:** Basic names the two things you're building (an embedding call, a similarity score) and the shape of a good test set. Intermediate gives the real formula and the exact API call, then asks what changes once this code runs on more than 5 sentences, in a program someone actually keeps running — batching every sentence into one API call instead of 5, and recognizing (and using) the fact that these particular vectors are already unit-length, instead of doing math that has no effect every single time.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

```
pick 5 sentences: sentence_1 and sentence_2 mean almost the same thing
                  sentence_3, sentence_4, sentence_5 are all unrelated

get an embedding (a list of numbers) for each sentence

make a function cosine_similarity(a, b) that returns how close two
    lists of numbers are

for every pair of sentences:
    compute cosine_similarity between their embeddings
    print the pair and the score

check: does the pair (sentence_1, sentence_2)
    score higher than every other pair?
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# embedding_similarity_practice.py
import math
from openai import OpenAI

client = OpenAI()

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
```

```
norm_a = math.sqrt(squares)
squares = 0.0
for x in b:
    squares = squares + x * x
norm_b = math.sqrt(squares)
return dot / (norm_a * norm_b)
```

**Expected output if you run just this (nothing calls these functions yet):** nothing — defining a function doesn't run it. Add your 5 sentences and the comparison loop below it to see any scores print.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

```
function get_embedding(text) -> list[float]:
    call the embeddings API, return response.data[0].embedding

function cosine_similarity(a, b) -> float:
    dot = sum of a[i] * b[i]
    norm_a = sqrt(sum of a[i]^2)
    norm_b = sqrt(sum of b[i]^2)
    return dot / (norm_a * norm_b)

main:
    sentences = [s1, s2, s3, s4, s5]    # s1, s2 are the similar pair
    embeddings = {s: get_embedding(s) for s in sentences}
    for each unique pair (s_a, s_b) from itertools.combinations(sentences, 2):
        score = cosine_similarity(embeddings[s_a], embeddings[s_b])
        print(s_a, s_b, score)
    confirm the (s1, s2) pair has the highest score of all pairs
```

```
# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI

client = OpenAI()

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
```

```

for x in b:
    squares = squares + x * x
norm_b = math.sqrt(squares)
return dot / (norm_a * norm_b)

```

Write your 5 sentences and the `combinations` -based comparison loop yourself, then compare both against the [Solution](#).

Once that's working, cut the unnecessary work:

```

sentences = [s1, s2, s3, s4, s5]  # s1, s2 are the similar pair

batch-embed every sentence in one API call:
response = client.embeddings.create(model=..., input=sentences)
embeddings = [item.embedding for item in response.data]  # same order

function fast_cosine_similarity(a, b) -> float:
    # a and b are already unit-length (this model guarantees it) --
    # skip both norm() calls entirely
    return sum of a[i] * b[i]

for each unique pair:
    score = fast_cosine_similarity(...)
    print the pair and score

sanity check once, don't just assume:
assert abs(math.sqrt(sum(x * x for x in embeddings[0])) - 1.0) < 1e-6

```

Here's almost the whole thing — fill in the missing piece yourself:

```

# embedding_similarity_practice.py
import math
from itertools import combinations
from openai import OpenAI

client = OpenAI()

SENTENCES: list[str] = [
    # your 5 sentences here
]

def get_embeddings_batch(texts: list[str]) -> list[list[float]]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=texts
    )

```

```

embeddings = []
for item in response.data:
    embeddings.append(item.embedding)
return embeddings

def fast_cosine_similarity(a: list[float], b: list[float]) -> float:
    # your turn: these vectors are already unit-length (verified below) --
    # just return the dot product, no norm() calls needed
    ...

# one-time sanity check that the "already unit-length" assumption really holds
# for this model, instead of trusting it blindly
def assert_unit_length(vector: list[float]) -> None:
    squares = 0.0
    for x in vector:
        squares = squares + x * x
    norm = math.sqrt(squares)
    assert abs(norm - 1.0) < 1e-6, f"expected a unit vector, got norm={norm}"

```

Fill in `fast_cosine_similarity`, embed `SENTENCES` in one batch call, run `assert_unit_length` on the first vector, then score every pair and compare all of your finished versions against the [Solution](#).

**Difference between Basic and Intermediate:** same underlying idea (embed, compare, sort by score) at 2 completeness levels. Basic calls the embeddings API once per sentence and uses the full cosine similarity formula every time, including the norm calculations. Intermediate also does the real work exactly once — one batched API call for all 5 sentences instead of 5 separate ones — and, having actually verified the vectors are unit-length instead of assuming it, drops the now-pointless norm division. Neither change affects the *answer* (the similar pair still scores highest) — they change how much unnecessary work the code does to get there, which only starts to matter once "5 sentences" becomes "5,000."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)